

Verified Attenuated Modus Ponens: A Depth-Bounded Safety Primitive for Contradiction-Tolerant AI Systems

Revision R1

Anonymous submission

Abstract

Deployed AI systems must often act while the normative ground truth is contested: cross-jurisdictional mandates conflict, clinical guidelines disagree, and regulatory guidance points in opposite directions. Classical reasoning explodes under contradiction, and existing mitigations provide no formal guarantee about behavior while the contradiction is unresolved. We prove a depth-bounded safety theorem for AI systems reasoning paraconsistently. Formulas carry graded contradiction values α in a bilattice semantics; high-stakes guard rules carry firing thresholds ϑ ; inference steps are governed by collapse classes with per-step contradiction tensions $T(H)$. We define VAMP (Verified Attenuated Modus Ponens), a calculus in which each inference step — of any arity — satisfies a local additive verification condition relating the contradiction grade of a conclusion to the grades of its premises, including its implication premise, and the tension of its governing class. The central result is a budget domination theorem: VAMP-valid proof objects admit a recursive contradiction budget that bounds formula-level grades at every node, so no high-stakes guard rule may fire at an action-licensing node whose budget remains below the minimum threshold across the guard-rule set. The audit is a single topological pass over the proof DAG: we show that the conservative tree-unfolded budget is computable on the DAG itself in linear time, so the conservatism costs certifiability, not runtime. The verification regime is motivated by a structural impossibility: max-form non-expansion fails for action-licensing implication under any consequent-monotone graded semantics. The audit does not restore that failed invariant; it replaces it with a per-step discipline whose violation is detectable, so that contradiction becomes a quantity that accumulates through inference at an audited rate. We provide a reference implementation with exact rational arithmetic: the worked examples are machine-checked, budget domination is validated on 10^4 random proof objects with full mutation detection, and an 80,000-node audit completes in about half a second. Scope is explicit: the theorem bounds derivability of high-stakes consequences relative to declared valuations and a faithful proof object; it does not bound real-world harm, and collapse-class selection, threshold setting, and valuation calibration remain normative-design choices outside the formalism. The contribution is a verifiable safety primitive for contradiction-tolerant AI systems and a governance

interface where commitments about contradiction tolerance become explicit, parameterized, and subject to oversight.

1 Introduction

Frontier AI systems increasingly act in domains where contradiction is not an aberration to be eliminated but a structural feature of the operating environment. A content moderation system operating across jurisdictions inherits incompatible legal mandates. A medical decision support system operating under conflicting clinical guidelines must reason despite, not around, the disagreement. An automated benefits-determination system encounters case law and regulatory guidance that point in opposite directions. In each case, the contradiction is not a sign that the AI system has malfunctioned — it is what the system was deployed to navigate.

Classical logic offers no traction in this regime: from $A \wedge \neg A$, anything follows, and any high-stakes action is licensed by any contradiction. Standard paraconsistent logics block global explosion but treat contradiction as binary — either present or not — and provide no quantitative handle on how much contradiction a particular inference is propagating. Existing AI safety mechanisms, including reinforcement learning from human feedback [Christiano et al., 2017] and constitutional AI [Bai et al., 2022], address contradiction only obliquely: they tune model behavior to avoid acting on contradictions detected in training, but offer no formal guarantee about behavior on novel contradictions encountered at deployment. None of these approaches gives an oversight body — a regulator, an internal review board, an affected stakeholder — a way to audit whether a particular system action was contradiction-driven and, if so, whether the contradiction was severe enough to warrant escalation.

This paper provides such an audit primitive.

We define VAMP (Verified Attenuated Modus Ponens), a local verification regime over proof objects in a paraconsistent calculus. VAMP assigns each proof node a computable contradiction budget $B(v)$, accumulating local contradiction grades through inference at a rate bounded by per-step collapse-class tensions. The central guarantee is operational: no high-stakes guard rule may fire at an action-licensing node whose budget remains below the minimum threshold across the guard-rule set. Audit reduces to a single topological pass

over the proof DAG, decidable in linear time over rational valuations — including for DAGs with heavy shared substructure, where the conservative tree-unfolded budget is computed without materializing the unfolding.

The contribution rests on a structural observation: the natural max-form non-expansion desideratum for paraconsistent inference — the requirement that contradiction in a derived formula be bounded by the maximum contradiction of its premises — fails for action-licensing implication under any consequent-monotone graded semantics. This is not a quirk of one particular t-norm; it is a feature of how implication compresses information about the consequent. Rather than weaken the calculus to recover non-expansion, we replace the desideratum with a per-step additive verification side condition. The negative result is not thereby converted into the failed invariant’s positive twin; instead, the unattainable invariant is replaced by an auditable discipline: contradiction becomes a quantity that accumulates through inference at a bounded, checked rate, and reasoning safety becomes a budget condition on that accumulation.

The paper makes five contributions:

1. We establish two impossibility results that motivate the framework. Łukasiewicz-style conjunctions with additive falsity accumulation (conjunctions whose falsity coordinates add rather than take maxima) violate max-form non-expansion (the requirement that a conclusion’s contradiction grade not exceed the maximum grade among its premises). More importantly, action-licensing implication violates max-form contradiction control even under conjunctions that behave non-expansively. The framework’s verification regime, VAMP, replaces the unattainable non-expansion invariant with auditable rate-limiting.

2. We prove a budget domination theorem: VAMP-valid proof objects admit a recursive contradiction budget $B_{\text{tree}}(v)$, defined per inference rule, that dominates the formula-level grade $\alpha(v)$ at every node. Chain derivations obey the bound $(2d+1)T(H)$ in general and $(d+1)T(H)$ under a declared contradiction-free rule base; binary proof trees admit the conservative worst-case bound $(2^{d+1}-1)T(H)$.

3. We specify VAMP as an audit procedure: a linear-time algorithm over finite proof DAGs with rational valuations that returns either a certified safety attestation or a flagged failure point, and fails closed — on rejection, the default action is suppressed and the case routes to a declared fallback. The procedure distinguishes descriptive nodes from terminal action-licensing nodes; reasoning may proceed through arbitrary contradiction budgets, but high-stakes action is gated at the threshold.

4. We provide a reference implementation with exact rational arithmetic and an empirical evaluation: the worked examples are machine-checked, budget domination is validated on 10^4 randomly generated proof objects with zero violations and 100% mutation detection, and audits of DAGs whose tree-unfoldings exceed 2^{4000} nodes complete in fractions of a second.

5. We illustrate the framework on cross-jurisdictional content moderation, demonstrating that the same default action

can be automatically certified in clean cases and routed to escalation in contradiction-laden ones, with the discrimination performed by the declared contradiction structure — precisely, by accumulated exposure to declared tension classes — rather than by case-by-case rule design.

The framework is explicit about its scope. The theorem bounds derivability of high-stakes consequences at bounded depth; it does not bound real-world harm. Collapse-class selection, threshold setting, and valuation calibration remain normative-design questions outside the formalism: the system designer must declare which collapse classes apply, what tensions they carry, what thresholds govern high-stakes guard rules, and where the valuations come from. VAMP makes these declarations legible — an explicit, auditable interface where governance choices about contradiction tolerance become subject to scrutiny — but the choices themselves are normative, not formal. The contribution is a verifiable safety primitive for contradiction-tolerant systems, not a theory of correct moral action.

We situate this contribution within an emerging regulatory landscape. The IEEE 7010 standard for well-being assessment of autonomous and intelligent systems [IEEE, 2020], the EU AI Act’s high-risk-system obligations under Articles 14 and 15 [EU, 2024], and the U.S. NIST AI Risk Management Framework [NIST, 2023] all require demonstrable mechanisms for human oversight of high-stakes AI actions. None of these instruments specifies what such mechanisms should formally guarantee. VAMP offers one: a decidable, local, depth-budgeted certificate that contradiction-driven action licensing has not occurred below threshold. Whether such certificates should be required, what threshold values are appropriate for which application domains, and how audit procedures interface with human review remain open governance questions. We do not resolve them. We make them tractable.

Detailed proofs of supporting lemmas, an additional implication-failure witness for the Łukasiewicz lift, the counterexample showing why naive DAG-aware budgets are unsound under VAMP, the proof that the tree-unfolded budget is computable directly on the DAG, and implementation details are provided in the Supplemental Materials.

2 Related Work

Paraconsistency and graded truth. The semantic substrate is standard: Belnap’s four-valued logic [Belnap, 1977], its bilattice generalizations [Ginsberg, 1988, Fitting, 1991, Arieli and Avron, 1996], da Costa’s C-systems [da Costa, 1974], and the logic-of-paradox tradition [Priest, 1979] establish that inference can survive contradiction; graded t-norm semantics [Hájek, 1998] supply the continuum of truth and falsity coordinates we use. VAMP takes this substrate as given and contributes none of it; the contribution is the verification and audit layer above it.

Annotated and possibilistic logic programming. The closest formal neighbors attach degrees to formulas and reason with them: paraconsistent logic programming [Blair and Subrahmanian, 1989], generalized annotated programs [Kifer and Subrahmanian, 1992], and possibilistic logic [Dubois and Prade, 1994], which tracks graded inconsistency of a knowledge base. These frameworks propagate annotations through inference; they do not impose per-step rate side conditions, do not define a certificate object separating certified from rejected derivations, and do not gate a distinguished class of action-licensing conclusions. VAMP’s delta is exactly that enforcement layer: local verification conditions, a budget recursion sound against a gauge semantics (Theorem 1), and a fail-closed audit.

Graded argumentation. Weighted argument systems [Dunne et al., 2011] and gradual argumentation semantics [Baroni et al., 2018] assign degrees to arguments under attack, and abstract argumentation [Dung, 1995] settles acceptability under conflict. These frameworks evaluate which conclusions survive a conflict; VAMP audits how much declared contradiction a specific proof object accumulates on the way to a specific action. The questions are complementary: an argumentation layer may adjudicate the knowledge base while VAMP gates the action taken from it.

Resource-bounded and substructural logics. Budgets invite comparison with linear logic’s resource discipline [Girard, 1987]. The analogy is real but partial: linear logic controls *use* of premises; the VAMP budget prices *contradiction flux* through steps and is tied to the gauge semantics by a soundness theorem rather than by a structural restriction on contexts.

Runtime verification, shielding, and enforcement. VAMP is naturally read as a runtime verification monitor [Leucker and Schallhart, 2009] whose monitored property is budget-boundedness of a proof object. In safe reinforcement learning, shields [Alshiekh et al., 2018] and the broader correct-by-construction runtime enforcement program [Könighofer et al., 2022] enforce temporal-logic safety over action sequences against an environment abstraction. VAMP differs on both axes: it requires no environment model and no temporal specification, and it certifies a property of the *reasoning artifact* behind a single high-stakes action rather than of the action sequence. The two compose: a shield can constrain what actions are executable while VAMP constrains which justifications may license them.

AI-safety training methods and their gap. RLHF [Christiano et al., 2017] and constitutional AI [Bai et al., 2022] shape dispositions during training; they provide no per-decision certificate. Calibration of learned confidence [Guo et al., 2017] and the faithfulness of chain-of-thought explanations [Turpin et al., 2023] are the two

assumptions a deployment of VAMP over learned components must discharge; we return to both in Sections 3 and 7.

3 Graded Bilattice Semantics

Fix a countable propositional language $\mathcal{L}$ closed under $\neg, \wedge, \vee, \rightarrow$. A *valuation* assigns each formula A a pair $v(A) = (\tau_A, \varphi_A) \in [0, 1]^2$, where τ_A measures graded support for A and φ_A graded support against A . We write the pair $v(A)$ to keep the formula A and its semantic value typographically distinct. The two coordinates are independent: evidence for and evidence against a claim may both be strong. This is the standard bilattice picture [Ginsberg, 1988, Fitting, 1991]; the classical corner points $(1, 0), (0, 1), (0, 0), (1, 1)$ recover Belnap’s four values [Belnap, 1977].

The *contradiction gauge* of A under a valuation is

$$\alpha(A) = g(\tau_A, \varphi_A) = \min(\tau_A, \varphi_A),$$

the extent to which A is simultaneously supported and denied. $\alpha(A) = 0$ iff at least one coordinate is null (no live contradiction); $\alpha(A) = 1$ iff A is fully supported and fully denied. Negation swaps coordinates: $v(\neg A) = (\varphi_A, \tau_A)$, whence $\alpha(\neg A) = \alpha(A)$ (Lemma 1, Supp. A) — a contradiction is a property of a claim and its denial jointly.

Conjunction and disjunction are interpreted by the Gödel ($\min/\max$) pair: $\tau_{A \wedge B} = \min(\tau_A, \tau_B)$, $\varphi_{A \wedge B} = \max(\varphi_A, \varphi_B)$, dually for $\vee$. Under this choice the gauge is non-expansive across $\wedge$ and $\vee$: $\alpha(A \wedge B) \leq \max(\alpha(A), \alpha(B))$, and dually (Lemma 2 and its corollary, Supp. A). The Łukasiewicz alternative, in which falsity accumulates additively across conjunction, already violates non-expansion at the level of $\wedge$ (Proposition 1); the Gödel choice is thus the conservative one, and the failure we exhibit for implication in Section 4 is not an artifact of an expansive conjunction.

Where valuations come from — and what is assumed.

The formalism is deliberately agnostic about the source of valuations: graded evidence pairs may be elicited from Dempster–Shafer belief functions [Shafer, 1976], from probabilistic ensembles, from calibrated classifier confidences [Guo et al., 2017], or from structured expert elicitation. This agnosticism is a scope boundary, not a solved problem. All guarantees in this paper are *relative to the declared valuations*: Theorem 1 certifies that a proof object’s conclusions carry bounded contradiction *given* the input grades; it is silent on whether those grades faithfully measure the world. Heterogeneous sources raise a genuine calibration problem — a 0.3 from a legal-conflict detector and a 0.3 from a clinical-guideline comparator need not be commensurable — and miscalibrated inputs yield certificates about the wrong quantity. We do not solve calibration here; we make the dependence explicit, and we identify calibration methodology for contradiction gauges as an open problem (Section 9). The

same holds for the tension function T and thresholds ϑ introduced below: they are declared normative-design parameters, and the audit makes them legible rather than correct.

4 The Failure of Max-Form Non-Expansion

The natural first desideratum for contradiction-tolerant inference is *max-form non-expansion*: derived formulas should be no more contradictory than the material they were derived from. Formally, for any derivation of $\Gamma \vdash B$ from a premise multiset $\Delta \subseteq \Gamma$,

$$\alpha(B) \leq \max_{A \in \Delta} \alpha(A). \quad (1)$$

Were (1) achievable, contradiction control would be a connective-level property: fix the semantics once, and every derivation inherits the bound. This section records that the desideratum is unattainable in the presence of action-licensing implication, for structural rather than semantic-parochial reasons. The propositions below are deliberately elementary; their role is architectural. They close off the connective-level route to safety and thereby motivate the per-step verification regime of Section 5.

Proposition 1 (Łukasiewicz conjunction expands the gauge). *Under the Łukasiewicz lift, where falsity accumulates additively across conjunction ($\varphi_{A \wedge B} = \min(1, \varphi_A + \varphi_B)$ with $\tau_{A \wedge B} = \max(0, \tau_A + \tau_B - 1)$), there exist valuations with $\alpha(A \wedge B) > \max(\alpha(A), \alpha(B))$.*

Proof. Take $v(A) = v(B) = (1, 0.4)$. Then $\alpha(A) = \alpha(B) = 0.4$, while $v(A \wedge B) = (1, 0.8)$, so $\alpha(A \wedge B) = 0.8$. $\square$

The Gödel lift avoids this failure at $\wedge$ and $\vee$ (Supp. A). The substantive failure survives the repair:

Proposition 2 (Independent grading defeats connective-level control). *Valuations assign pairs to formulas independently; in particular there exist valuations with $\alpha(A) = 0$, $\alpha(A \rightarrow B) = 0$, and $\alpha(B)$ arbitrary in $[0, 1]$.*

Proposition 3 (Max-form failure for graded implication). *Let $\Rightarrow$ be any graded implication whose falsity coordinate is monotone non-decreasing in the falsity coordinate of its consequent. Then detachment cannot satisfy (1): there exist valuations with $\alpha(A) = \alpha(A \Rightarrow B) = 0$ while $\alpha(B) > 0$.*

Proof sketch. Set $v(A) = (1, 0)$ and $v(B) = (1, c)$ for any $c > 0$ with $\tau_B = 1$, so $\alpha(B) = c$. Monotonicity constrains $\varphi_{A \Rightarrow B}$ from below by a function of φ_B that vanishes when the implication’s truth coordinate saturates; the witness in Supp. B exhibits, for both the Gödel and Łukasiewicz implications, valuations with $\alpha(A) = \alpha(A \Rightarrow B) = 0$ and $\alpha(B) = c$. The premises of detachment then carry zero gauge while the conclusion carries c . $\square$

Remark 1 (What these propositions do and do not say). Propositions 2 and 3 record an observation, not a deep theorem: when the two coordinates of a valuation are assigned per formula, no consequent-monotone conditional can force the gauge of a detached consequent to track the gauges of the detachment’s premises. The reason is structural: the implication’s own value compresses away the information about φ_B that (1) would need. We state the propositions because of what they rule out — any hope that choosing the right connectives yields contradiction control for free — not because their proofs are demanding.

Remark 2 (Schemas, not conditionals). Action-licensing rules — “if conditions C obtain, action a is licensed” — are inference schemas in this framework, not object-language implications. The impossibility nonetheless bears on them: any attempt to internalize such a rule as a consequent-monotone graded conditional inherits the failure of Proposition 3, so connective-level semantics cannot bound the contradiction of what the rule licenses. This is why the calculus of Section 5 treats action licensing by explicit schemas carrying declared thresholds and additive verification conditions, rather than by any choice of graded $\rightarrow$.

The pivot the failure forces is conceptual. Contradiction is not a quantity that can be prevented from growing; it is a quantity that *accumulates* through inference. The design question becomes: at what rate, and audited how? Section 5 answers with local, per-step verification conditions; Section 6 shows the local conditions compose into a global bound. The reason the additive form is the right side condition is structural: what implication destroys is exactly the max-form comparison, and what survives is an additive comparison against the step’s declared tension.

5 The LPA- α Calculus

5.1 Collapse Classes and Tension

A *collapse-class assignment* is a function H from inference steps to a finite set $\mathcal{H}$ of collapse classes, together with a *tension function* $T : \mathcal{H} \rightarrow \mathbb{Q} \cap [0, 1]$. The class $H(s)$ of a step s records which declared contradiction — which pair of conflicting authorities, guidelines, or mandates — the step reasons across; $T(H(s))$ is the declared per-step price of doing so. Steps that reason within a single coherent authority carry a class of tension 0. Collapse-class selection and tension assignment are normative-design declarations (Section 3); the calculus consumes them, audits against them, and makes them legible.

5.2 Inference Rules

The calculus has two graded rules plus the classical structural rules. Both graded rules carry verification conditions of the same shape, stated here in words to fix the reading: *each rule bounds the conclusion’s gauge by the sum of the gauges*

of its premises — for VAMP, assertion premises and implication premise together — plus the tension of the step’s collapse class. The per-rule forms follow.

VAMP (Verified Attenuated Modus Ponens), arity $m \geq 1$. From $\Gamma \vdash A_1, \dots, \Gamma \vdash A_m$ and $\Gamma \vdash (\bigwedge_{j=1}^m A_j) \rightarrow B$, infer $\Gamma \vdash B$, subject to:

- *Activation:* $\alpha(A_j) < \eta$ for every assertion premise $j = 1, \dots, m$, where η is the declared activation ceiling;
- *Verification:*

$$\alpha(B) \leq \sum_{j=1}^m \alpha(A_j) + \alpha\left(\bigwedge_j A_j \rightarrow B\right) + T(H(s)). \quad (2)$$

For $m = 1$ this is attenuated modus ponens in its familiar binary form: from A and $A \rightarrow B$, infer B with $\alpha(B) \leq \alpha(A) + \alpha(A \rightarrow B) + T(H(s))$. The verification condition is the local rate-limiting that Section 4 established as necessary: the conclusion’s gauge may exceed the *sum* of the premises’ gauges — assertions and implication together — by at most the step’s collapse-class tension. By Gödel non-expansion (Supp. A, Lemma 2), the bundled antecedent satisfies $\alpha(\bigwedge_j A_j) \leq \max_j \alpha(A_j) < \eta$ whenever activation holds, so generalizing the arity does not disturb the activation semantics.

Rule regimes. The implication premise of a VAMP step is a proof node like any other, and its gauge appears in (2). A theory declares one of two regimes for its rule base $\mathcal{R}_{\text{imp}}$ of licensed implications. Under the *audited-rule regime*, implications carry their own declared or derived gauges, which enter verification and budgets. Under the *certified-rule regime*, the designer certifies $\alpha(w) = 0$ for every rule-base implication w — a declaration that the rules themselves are not in contradiction — and their contributions vanish. The declaration is part of the audit input and is itself an auditable commitment; a theory that cannot certify its rules uses the audited regime and pays for rule contradiction explicitly. Corollary 1 states the depth bound under both regimes.

VGP (Verified Guarded Paraconsistent consequence). High-stakes consequences of a live contradiction are licensed only through guard schemas. For a consequence schema R with declared threshold $\vartheta(R)$: from $\Gamma \vdash A$ and $\Gamma \vdash \neg A$, infer $\Gamma \vdash R(A)$, subject to:

- *Activation:* $\alpha(A) \geq \vartheta(R)$ — the guard fires only on contradictions at least as severe as its threshold;
- *Schema-boundedness:* $\alpha(R(A)) \leq \alpha(A) + T(H(s))$.

By negation preservation the pair $(A, \neg A)$ carries a single gauge, so the schema-boundedness condition charges it once.

Action expressions and type discipline. For each schema R and formula A , the expression $R(A)$ belongs to an action language $\mathcal{L}_{\text{act}}$ disjoint from $\mathcal{L}$. Valuations extend to $\mathcal{L}_{\text{act}}$: $v(R(A))$ is supplied by the same exogenous valuation source as ordinary formulas and is read as graded support for and against *performing* the licensed action given the record. When a deployment has no independent assessment of the action, the canonical default is to inherit the trigger’s pair, $v(R(A)) := v(A)$, which satisfies schema-boundedness with slack $T \geq 0$. Action expressions are *terminal*: no VAMP or VGP step may take an $\mathcal{L}_{\text{act}}$ expression as a premise — they are licenses, not propositions — and correspondingly action-licensing nodes are sinks of every well-formed proof object (Definition 4). If a deployment observes the outcome of a taken action, that observation re-enters as a fresh premise node with its own valuation in a subsequent proof object; it does not flow through the action node.

Structural rules. Identity, weakening, and cut are admitted classically at the sequent level. Audit-relevant proof objects (Definition 3) consist of premise, VAMP, and VGP nodes only; structural steps neither create budgeted nodes nor carry tension.

5.3 Coordination and Deployment Semantics

Definition 1 (Coordination). Let $\mathcal{R}_{\text{high}} \neq \emptyset$ be the declared set of high-stakes guard schemas and $\vartheta^* = \min_{R \in \mathcal{R}_{\text{high}}} \vartheta(R)$. A theory is *coordinated* with margin $\delta > 0$ if $\eta + \delta \leq \vartheta^*$.

Coordination separates the regimes: VAMP propagates only material whose gauge is strictly below η , while high-stakes guards fire only at or above $\vartheta^* \geq \eta + \delta$. The nonemptiness requirement on $\mathcal{R}_{\text{high}}$ is part of the definition — with no declared high-stakes schemas the minimum is vacuous ($+\infty$) and the coordination condition would hold trivially while guarding nothing.

Definition 2 (Dead zone and fail-closed deployment). Grades in the interval $[\eta, \vartheta^*)$ are in the *dead zone*: too contradicted for VAMP to propagate, not contradicted enough to fire any high-stakes guard. The operational semantics is fail-closure. A VAMP-deployed system executes the default action licensed at a node $v \in V_{\text{act}}$ *iff* the audit returns *certified*; on *rejected* — including every case in which a step’s trigger grade falls in the dead zone, since activation then fails — the default action is suppressed and the case routes to a declared fallback $F \in \{\text{escalate-to-human-review, refuse, defer}\}$ fixed as part of the theory. *certified* and *rejected* exhaust the audit’s outcomes; there is no silent pass-through.

The dead zone is thus not an unspecified gap but the calculus’s designed escalation region: material too contested to reason from automatically, yet below the declared severity at which contradiction-triggered guards act, is exactly the material a fail-closed deployment sends to its fallback.

Definition 3 (Proof object). A *proof object* π is a finite DAG whose nodes are labeled premise, VAMP, or VGP; each VAMP node of arity m has m assertion in-edges and one implication in-edge; each VGP node has the pair $(A, \neg A)$ as in-edges; a subset V_{act} of nodes is flagged action-licensing.

Definition 4 (Well-formedness). A proof object is *well-formed* for a theory when (i) every step’s activation and verification conditions hold under the declared valuations; (ii) $\mathcal{R}_{\text{high}} \neq \emptyset$ and the theory is coordinated (Definition 1); (iii) every $v \in V_{\text{act}}$ is a sink; (iv) the declared rule regime, tension function, thresholds, and fallback F are supplied.

6 Contradiction Budgets and the Safety Theorem

Definition 5 (Rule-aware contradiction budget). Let π be a well-formed proof object. The *tree-unfolded budget* B_{tree} is defined by recursion over π . For a premise node, $B_{\text{tree}}(v) = \alpha(v)$. For a VAMP _{m} node v with assertion premises $u_1, \dots, u_m$ and implication premise w at step s ,

$$B_{\text{tree}}(v) = \sum_{j=1}^m B_{\text{tree}}(u_j) + B_{\text{tree}}(w) + T(H(s)).$$

For a VGP node v over the pair (u, u') at step s ,

$$B_{\text{tree}}(v) = \min(B_{\text{tree}}(u), B_{\text{tree}}(u')) + T(H(s)).$$

The design principle is that *the budget recursion is the verification condition with gauges replaced by budgets*. VAMP’s verification bound sums all premises, so its budget does; VGP’s bound charges the contradictory pair once (negation preservation), so its budget takes the tighter sound choice, the minimum over the pair — soundness of that choice is exactly the content of the inductive step in Theorem 1, and its supp counterpart (Supp. A, Lemma 3) is stated in the same convention.

Theorem 1 (Budget domination). *Let π be a well-formed proof object under declared valuations, tensions, and thresholds. Then $\alpha(v) \leq B_{\text{tree}}(v)$ for every node v of π .*

Proof. Induction over a topological order of π . *Premise:* $\alpha(v) = B_{\text{tree}}(v)$ by definition. *VAMP _{m} :* by verification (2) and the inductive hypothesis at each premise, $\alpha(v) \leq \sum_j \alpha(u_j) + \alpha(w) + T(H(s)) \leq \sum_j B_{\text{tree}}(u_j) + B_{\text{tree}}(w) + T(H(s)) = B_{\text{tree}}(v)$. *VGP:* write A for the trigger; by negation preservation $\alpha(A) = \alpha(\neg A)$, and by the inductive hypothesis applied to both members of the pair, $\alpha(A) \leq \min(B_{\text{tree}}(u), B_{\text{tree}}(u'))$; schema-boundedness then gives $\alpha(v) \leq \alpha(A) + T(H(s)) \leq B_{\text{tree}}(v)$. $\square$

The proof is a direct induction, and deliberately so: the verification conditions were designed so that local checking telescopes into a global bound. We return to what this does and does not establish at the end of the section.

Remark 3 (Closed form). For proof objects containing only premise and VAMP nodes, unwinding the recursion gives

$$B_{\text{tree}}(v) = \sum_{\ell \in \text{leaves}(\mathcal{U}(v))} \alpha(\ell) + \sum_{s \in \text{steps}(\mathcal{U}(v))} T(H(s)),$$

where $\mathcal{U}(v)$ is the tree unfolding of v ’s derivation: premise mass plus tension mass, with a shared ancestor’s contribution duplicated across each distinct path — counted once per path, not once per node. With VGP nodes the identity relaxes to $\leq$ under the min convention. The budget of a certified derivation is therefore precisely its accumulated exposure: to premise contradiction and to declared tension classes.

Corollary 1 (Chain bound, both rule regimes). *Let $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_d$ be a chain of arity-1 VAMP steps in class H with implication premises $w_i = (v_i \rightarrow v_{i+1})$. Then*

$$B_{\text{tree}}(v_d) = \alpha(v_0) + \sum_{i=0}^{d-1} \alpha(w_i) + dT(H).$$

Under the audited-rule regime with all premises bounded by the tension — $\alpha(v_0) \leq T(H)$ and $\alpha(w_i) \leq T(H)$ for all i — this gives $B_{\text{tree}}(v_d) \leq (2d + 1)T(H)$. Under the certified-rule regime ($\alpha(w_i) = 0$) with $\alpha(v_0) \leq T(H)$ it gives $B_{\text{tree}}(v_d) \leq (d + 1)T(H)$.

Corollary 2 (Binary-tree worst case). *Under the certified-rule regime, a balanced binary proof tree of depth d in class H with premise gauges $\leq a$ satisfies $B_{\text{tree}} \leq 2^d a + (2^d - 1)T(H)$; with $a \leq T(H)$, $B_{\text{tree}} \leq (2^{d+1} - 1)T(H)$. Under the audited-rule regime the per-level rule gauges add to the same recursion.*

The certifiability horizon. The exponential in Corollary 2 is a bound on the budget’s *value*, and Section 7 shows it costs nothing computationally. What it does cost is certifiability, and the cost has a closed form. A chain certifies at its action node iff $\alpha(v_0) + \sum_i T(H(s_i)) < \vartheta^*$ (certified-rule regime): the certifiable tension mass along a chain is $\vartheta^* - \alpha(v_0)$, so with uniform tension T at most $\lceil \vartheta^*/T \rceil - 1$ high-tension steps are certifiable. A balanced binary tree of depth d with premise gauges $\leq a$ certifies iff $2^d a + (2^d - 1)T < \vartheta^*$, i.e., $d < \log_2((\vartheta^* + T)/(a + T))$. Deep derivations with heavy reuse of contested material therefore exhaust their budgets quickly — by design: each additional use along a distinct path re-exposes the conclusion to the same declared contradiction, and the counterexample in Supp. C shows that any budget charging shared ancestors once per node is unsound for VAMP. Whether real deployed proof objects sit inside or outside the horizon is an empirical question about those deployments, and we make no claim about its typical answer; the horizon formulas are the design interface through which T and ϑ^* should be chosen for an expected proof-depth profile. A budget-tightening *rule* refinement that is sharing-aware without unsoundness remains open (Section 9).

What Theorem 1 does and does not establish. It does establish: (i) *locality to globality* — per-step checkable side conditions compose into a bound at every node, so a linear scan suffices for a global property; (ii) *decidability* — over rational inputs the bound is exactly computable, yielding certificates rather than estimates; (iii) *fail-closed gating* — combined with coordination, no high-stakes guard fires at an action node whose budget is below ϑ^* . It does not establish: mathematical depth (the induction is one line per rule, by design); anything about derivations outside VAMP-validity (the audit’s answer there is **rejected**, not a bound); anything about real-world harm; and nothing about the fidelity of the inputs — the certificate is relative to declared valuations, tensions, and a proof object faithful to the reasoning that produced the action (Sections 3 and 7). The theorem does not convert the impossibility of Section 4 into its positive twin; it replaces an unattainable invariant with an auditable discipline.

7 VAMP as an Audit Procedure

7.1 Inputs and Decidability

The audit consumes: a proof object π (n nodes, e edges); gauges $\alpha : V \rightarrow \mathbb{Q} \cap [0, 1]$; the tension function T ; thresholds ϑ with $\mathcal{R}_{\text{high}} \neq \emptyset$; the activation ceiling η ; the declared rule regime and fallback F . The restriction from real-valued semantics to rational inputs is what makes auditing exactly decidable: $\mathbb{Q} \cap [0, 1]$ admits finite representation and exact comparison, so every check below is a terminating computation rather than an approximation. The semantics of Section 3 remains over $[0, 1]$; the audit operates on the rational-valued inputs a deployment actually supplies.

7.2 Algorithm

Proposition 4 (Audit soundness). *If VAMP-AUDIT returns **certified**, then π is well-formed, every step’s conditions hold, $\alpha(v) \leq \beta[v]$ at every node (Theorem 1), and no high-stakes guard rule fires at any action-licensing node: every $v \in V_{\text{act}}$ has $\beta[v] < \vartheta^*$, hence $\alpha(v) < \vartheta^* \leq \vartheta(R)$ for every $R \in \mathcal{R}_{\text{high}}$, so no VGP activation over v ’s license can be met.*

Lemma 1 (DAG evaluation computes the tree-unfolded budget). *Let β be computed by the memoized topological pass of Algorithm 1 on the proof DAG π . Then $\beta[v] = B_{\text{tree}}(v)$ for every node v , where B_{tree} is Definition 5’s recursion over the tree unfolding of v ’s derivation.*

Proof idea. Induction over topological order: the unfolding of v is the step at v applied to the unfoldings of its premises, and Definition 5’s recursion is compositional in exactly that structure, so the memoized value at each premise is the unfolded value there. Full proof in Supp. C. $\square$

Proposition 5 (Complexity: linear on the DAG). *VAMP-AUDIT performs $O(n+e)$ arithmetic operations and comparisons on the proof DAG itself, for arbitrary DAGs including those with shared substructure; no unfolding is constructed.*

Algorithm 1: VAMP-AUDIT (single pass, DAG-native)

Input: $\pi, \alpha, T, \vartheta, \mathcal{R}_{\text{high}}, \eta$, regime, fallback F . **Output:** $\kappa \in \{\text{certified}, \text{rejected}\}$; step report σ ; budgets β ; flags ρ .

1. *Well-formedness:* check $\mathcal{R}_{\text{high}} \neq \emptyset$; $\eta + \delta \leq \vartheta^* := \min_{R \in \mathcal{R}_{\text{high}}} \vartheta(R)$; every $v \in V_{\text{act}}$ is a sink. On failure: $\kappa \leftarrow \text{rejected}$, engage F .
2. *Topological pass:* for each node v in topological order: premise: $\beta[v] \leftarrow \alpha(v)$. VAMP_m step: check activation ($\alpha(u_j) < \eta$, all j) and verification (2); on pass, $\beta[v] \leftarrow \sum_j \beta[u_j] + \beta[w] + T(H(s))$; on fail, record $\sigma[s]$, $\kappa \leftarrow \text{rejected}$, engage F . VGP step: check $\alpha(A) = \alpha(\neg A)$, activation ($\alpha(A) \geq \vartheta(R)$), schema-boundedness; on pass, $\beta[v] \leftarrow \min(\beta[u], \beta[u']) + T(H(s))$; on fail, as above.
3. *High-stakes scan:* for each v : $\rho[v] \leftarrow \text{below}$ if $\beta[v] < \vartheta^*$ else **at-or-above**. If any $v \in V_{\text{act}}$ is **at-or-above**: $\kappa \leftarrow \text{rejected}$, engage F . Else $\kappa \leftarrow \text{certified}$.

Figure 1: The revised audit. No tree unfolding is materialized; step 2 computes B_{tree} on the DAG itself (Lemma 1).

*The exponential growth of Corollary 2 concerns the value of B_{tree} — hence certifiability — not the audit’s running time.*¹

This corrects, and strengthens, the complexity claim of the original version of this paper, which asserted linearity in the size of the unfolded object: by Lemma 1 the unfolded object never needs to exist.

7.3 Descriptive versus Action-Licensing Nodes

Reasoning and acting are gated differently. Descriptive nodes may carry budgets at or above ϑ^* ; the audit flags them (ρ) but does not reject on their account — a system may *know about* severe contradictions, and derivations that merely describe the conflict are the raw material of escalation reports. Rejection is triggered only at action-licensing nodes, which are terminal (Definition 4): the license is the end of the audited derivation, not a premise for further reasoning, and any observed consequence of a taken action re-enters subsequent proof objects as a fresh premise with its own valuation. The gauge of an action expression itself is supplied like any other valuation, with $v(R(A)) := v(A)$ as the canonical default (Section 5); the audit checks schema-boundedness rather than deriving the value.

7.4 Worked Example: Cross-Jurisdictional Moderation

A moderation system operates across jurisdictions J_1 (content of type X must be removed) and J_2 (content of type X

¹Bit-complexity: with all inputs over a common denominator D , every budget is a multiple of $1/D$ bounded by $2^{O(d)} \max(1, T_{\text{max}})$ at depth d , so operands carry $O(d + \log D)$ bits and the audit costs $O((n+e)(d + \log D))$ bit operations. Section 8 confirms the linear growth empirically.

is protected). The theory declares: collapse classes H_{plain} ($T = 0$) for within-jurisdiction steps and $H_{J_1 J_2}$ ($T = 0.30$) for steps reasoning across the conflict; activation ceiling $\eta = 0.5$; one high-stakes guard schema (*escalate*) with $\vartheta = 0.60$, so $\mathcal{R}_{\text{high}} \neq \emptyset$ and $\vartheta^* = 0.60$; coordination margin $\delta = 0.1$ ($\eta + \delta = 0.6 \leq \vartheta^*$); fallback $F = \text{escalate-to-human-review}$; the *certified-rule regime* — every rule-base implication w_i is declared with $\alpha(w_i) = 0$.

Contradiction-laden variant. Premises: P_1 (the item matches J_1 ’s removal predicate, $\alpha = 0.05$), P_2 (it matches J_2 ’s protection predicate, $\alpha = 0.05$), P_3 (both jurisdictions’ mandates apply to this item, $\alpha = 0$). The proof object:

- s_1 : VAMP₁ from P_1, w_1 in H_{plain} , deriving P_4 (J_1 requires removal), $\alpha(P_4) = 0.05$. Verification: $0.05 \leq 0.05 + 0 + 0$. Budget $\beta(P_4) = 0.05$.
- s_2 : VAMP₁ from P_2, w_2 , deriving P_5 (J_2 forbids removal), $\alpha(P_5) = 0.05$; $\beta(P_5) = 0.05$.
- s_3 : VAMP₃ from P_4, P_5, P_3 and $w_3 = ((P_4 \wedge P_5 \wedge P_3) \rightarrow P_6)$ in $H_{J_1 J_2}$, deriving P_6 (the item is under actively conflicting mandates), $\alpha(P_6) = 0.30$. Activation: $\max(0.05, 0.05, 0) < 0.5$. Verification: $0.30 \leq 0.05 + 0.05 + 0 + 0 + 0.30$. Budget $\beta(P_6) = 0.05 + 0.05 + 0 + 0 + 0.30 = 0.40$.
- s_4 : VAMP₁ from P_6, w_4 in $H_{J_1 J_2}$, deriving the action-licensing node P_7 (default action: automatic removal), $\alpha(P_7) = 0.55$. Activation: $0.30 < 0.5$. Verification: $0.55 \leq 0.30 + 0 + 0.30$. Budget $\beta(P_7) = 0.40 + 0 + 0.30 = 0.70$.

$P_7 \in V_{\text{act}}$ is a sink; $\beta(P_7) = 0.70 \geq \vartheta^* = 0.60$, so $\rho(P_7) = \text{at-or-above}$ and the audit returns **rejected**: automatic removal is suppressed and the case routes to human review. Note what performed the discrimination: two steps of exposure to the declared J_1 – J_2 tension (0.30 each) on top of 0.10 of premise mass.

Clean variant. Same premises and rules, but the derivation to the action node crosses the conflict once: s'_3 derives the action-licensing node P'_6 (remove, with routine cross-jurisdiction handling), $\alpha(P'_6) = 0.20$, in $H_{J_1 J_2}$. Budget $\beta(P'_6) = 0.40 < 0.60$: **certified**, and automatic action proceeds. One exposure to the declared tension is within budget; two are not — at these declared parameters, that is exactly the policy the designer wrote down, now enforced and legible. Both variants are machine-checked by the reference implementation (Section 8).

7.5 What the Certificate Discriminates

The budget of a certified-regime derivation is its premise mass plus its tension mass (Remark 3). The certificate therefore discriminates by *accumulated exposure to declared contradiction*: counting, weighted by T , how many times a derivation reasons across each declared conflict, duplicated

across each distinct path of reuse. We state this plainly because it is the mechanism, not a hidden simplification: the framework’s claim is not that tension exposure is a semantically deep quantity, but that it is sound to account for it this way (Theorem 1), that nothing tighter survives the counterexample of Supp. C under this calculus, and that the resulting certificates are local, decidable, and legible to oversight. Whether exposure-counting with declared tensions is an adequate operationalization of “contradiction-driven” for a given domain is a normative-design question the framework surfaces rather than settles.

7.6 Deployment Architectures: Who Constructs the Proof Object?

The audit consumes a typed proof object; three architectures supply one. (i) *Native symbolic and neuro-symbolic pipelines* — rule engines, planners, retrieval-plus-rules systems — already emit typed inference steps; VAMP is a wrapper on their traces. (ii) *Post-hoc reconstruction* from a language model’s chain-of-thought: a parser maps the trace to premise/VAMP/VGP steps, and the certificate is then a statement about the reconstruction. This inherits the faithfulness problem — model-produced rationales can misreport the computation that produced the answer [Turpin et al., 2023] — so a certificate over a reconstructed trace certifies the trace, not the model’s underlying process; we flag this assumption rather than discharge it. (iii) *Gated hybrids*: a learned component proposes, a symbolic layer expresses the license as a proof object, and only certified licenses execute — the architecture our fail-closed semantics (Definition 2) anticipates. Constructing faithful proof objects from unconstrained LLM reasoning is an open problem we name, not solve (Section 9).

8 Reference Implementation and Empirical Checks

We implement Algorithm 1 in Python with exact rational arithmetic (`fractions.Fraction`; `vamp_audit.py`, ~250 lines) and report four checks. All numbers below are reproduced by `experiments.py` (seed 396); none involve a live reasoning system — this is a reference implementation with synthetic evaluation, offered as evidence of implementability and of the complexity claims, not of integration.

E1: machine-checked examples. Both variants of Section 7.4 audit as stated ($\beta(P_7) = 7/10$, **rejected**; $\beta(P'_6) = 2/5$, **certified**), and the Supp. C counterexample verifies exactly: with $\varepsilon = 0.1$, $t = 0.2$, the realized gauge at the merge node is $\alpha(v) = 0.8$, exceeding the naive DAG budget 0.7 while equal to the tree-unfolded budget 0.8.

E2: budget domination and mutation detection. On 10^4 randomly generated VAMP-valid proof objects (119,876 nodes; mixed arities, mixed rule regimes, random tension

placement), $\alpha(v) \leq \beta[v]$ held at every node: zero violations, as Theorem 1 requires. On 2,000 mutants — one derived node’s gauge pushed 0.01 above its verification bound — the audit flagged 2,000/2,000 (100% detection): the certificate is falsifiable and the checks are load-bearing.

E3: runtime is linear on the DAG. Width scaling at fixed depth: 800-, 8,000-, and 80,000-node DAGs audit in 4.9 ms, 42.6 ms, and 533 ms respectively ($\approx 5\text{--}7 \mu\text{s}$ per node) on a single commodity core. Depth scaling on diamond-ladder DAGs with maximal sharing — where the tree unfolding is exponential — confirms Proposition 5 and the footnoted bit-cost:

depth	DAG nodes	unfolding	time (s)	budget bits
100	601	$\sim 2^{102}$	0.009	106
1000	6001	$\sim 2^{1002}$	0.036	1006
4000	24001	$\sim 2^{4002}$	0.162	4006

An audit over a DAG whose unfolding has more nodes than 2^{4000} completes in a sixth of a second; budget numerators grow linearly in depth, exactly as the bit-complexity analysis predicts.

E4: the certifiability horizon, empirically. On 4-step chains under the certified-rule regime with $T = 0.3$, $\vartheta^* = 0.6$, and premise mass 0.05: zero or one high-tension steps certify ($\beta = 0.05, 0.35$); two or more reject ($\beta = 0.65, 0.95, 1.25$) — the once-certified/twice-rejected arithmetic of Section 7.4, now as a sweep. On random DAGs (audited-rule regime, half the steps high-tension), certification rates fall from 1.00 at depth 1 to 0.58 at depth 8, tracing the horizon of Section 6 as accumulated tension mass approaches ϑ^* .

9 Conclusion

Contradiction-tolerant reasoning needs a safety story that survives contact with the impossibility of connective-level control. VAMP’s story: contradiction accumulates through inference at an audited rate; budgets computed in one linear pass over the proof DAG dominate gauges everywhere; high-stakes action is gated, fail-closed, at a declared threshold. The revision strengthens the computational claim — the conservative tree-unfolded budget is computable without unfolding — and delivers a reference implementation whose checks are falsifiable and checked.

Open problems, named as such: (i) a sharing-sensitive budget *rule* that tightens the tree-unfolded account without the unsoundness of naive per-node counting (the value computation is now known to be cheap; the open question is a tighter sound recursion); (ii) calibration methodology for contradiction gauges across heterogeneous sources, without which certificates measure declared rather than actual contradiction; (iii) faithful construction of proof objects from unconstrained LLM reasoning, without which post-hoc certificates attach to

reconstructions; (iv) verification conditions for graded implication introduction beyond consequent-monotone forms; (v) completeness: characterizing exactly which gauge profiles are VAMP-certifiable at given (T, η, ϑ^*) .

The governance interface stands: collapse classes, tensions, thresholds, rule regimes, and fallbacks are declarations an oversight body can read, contest, and re-parameterize, and the certificate is a decidable object an auditor can recompute. What VAMP contributes is not the resolution of contested normative ground but a mechanism by which acting across it becomes explicit, priced, and reviewable.

References

- [Alshiekh et al., 2018] Alshiekh, M.; Bloem, R.; Ehlers, R.; Könighofer, B.; Niekum, S.; and Topcu, U. 2018. Safe Reinforcement Learning via Shielding. In *Proc. AAAI-18*.
- [Arieli and Avron, 1996] Arieli, O., and Avron, A. 1996. Reasoning with Logical Bilattices. *Journal of Logic, Language and Information* 5(1):25–63.
- [Bai et al., 2022] Bai, Y., et al. 2022. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073.
- [Baroni et al., 2018] Baroni, P.; Rago, A.; and Toni, F. 2018. How Many Properties Do We Need for Gradual Argumentation? In *Proc. AAAI-18*.
- [Belnap, 1977] Belnap, N. D. 1977. A Useful Four-Valued Logic. In *Modern Uses of Multiple-Valued Logic*, 5–37. Reidel.
- [Blair and Subrahmanian, 1989] Blair, H. A., and Subrahmanian, V. S. 1989. Paraconsistent Logic Programming. *Theoretical Computer Science* 68(2):135–154.
- [Christiano et al., 2017] Christiano, P., et al. 2017. Deep Reinforcement Learning from Human Preferences. In *Proc. NeurIPS*.
- [da Costa, 1974] da Costa, N. C. A. 1974. On the Theory of Inconsistent Formal Systems. *Notre Dame Journal of Formal Logic* 15(4):497–510.
- [Dubois and Prade, 1994] Dubois, D., and Prade, H. 1994. Possibilistic Logic. In *Handbook of Logic in Artificial Intelligence and Logic Programming*, Vol. 3. Oxford University Press.
- [Dung, 1995] Dung, P. M. 1995. On the Acceptability of Arguments and its Fundamental Role in Nonmonotonic Reasoning, Logic Programming and n-Person Games. *Artificial Intelligence* 77(2):321–357.
- [Dunne et al., 2011] Dunne, P. E.; Hunter, A.; McBurney, P.; Parsons, S.; and Wooldridge, M. 2011. Weighted Argument Systems: Basic Definitions, Algorithms, and Complexity Results. *Artificial Intelligence* 175(2):457–486.
- [EU, 2024] European Union. 2024. Regulation (EU) 2024/1689 (Artificial Intelligence Act), Articles 14–15.
- [Fitting, 1991] Fitting, M. 1991. Bilattices and the Semantics of Logic Programming. *Journal of Logic Programming* 11(2):91–116.

- [Ginsberg, 1988] Ginsberg, M. L. 1988. Multivalued Logics: A Uniform Approach to Reasoning in Artificial Intelligence. *Computational Intelligence* 4:265–316.
- [Girard, 1987] Girard, J.-Y. 1987. Linear Logic. *Theoretical Computer Science* 50(1):1–102.
- [Guo et al., 2017] Guo, C.; Pleiss, G.; Sun, Y.; and Weinberger, K. Q. 2017. On Calibration of Modern Neural Networks. In *Proc. ICML*.
- [Hájek, 1998] Hájek, P. 1998. *Metamathematics of Fuzzy Logic*. Kluwer.
- [IEEE, 2020] IEEE. 2020. IEEE 7010-2020: Recommended Practice for Assessing the Impact of Autonomous and Intelligent Systems on Human Well-Being.
- [Kifer and Subrahmanian, 1992] Kifer, M., and Subrahmanian, V. S. 1992. Theory of Generalized Annotated Logic Programming and its Applications. *Journal of Logic Programming* 12(3–4):335–367.
- [Könighofer et al., 2022] Könighofer, B., et al. 2022. Correct-by-Construction Runtime Enforcement in AI – A Survey. In *Principles of Systems Design*, LNCS 13660. Springer.
- [Leucker and Schallhart, 2009] Leucker, M., and Schallhart, C. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78(5):293–303.
- [NIST, 2023] National Institute of Standards and Technology. 2023. AI Risk Management Framework (AI RMF 1.0).
- [Priest, 1979] Priest, G. 1979. The Logic of Paradox. *Journal of Philosophical Logic* 8(1):219–241.
- [Shafer, 1976] Shafer, G. 1976. *A Mathematical Theory of Evidence*. Princeton University Press.
- [Turpin et al., 2023] Turpin, M.; Michael, J.; Perez, E.; and Bowman, S. R. 2023. Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *Proc. NeurIPS*.